

分治之倍增题目选讲

北京大学 王欣灏

洛谷1613 跑路

- 小A的工作不仅繁琐，更有苛刻的规定，要求小A每天早上在6:00之前到达公司，否则这个月工资清零。可是小A偏偏又有赖床的坏毛病。于是为了保住自己的工资，小A买了一个十分牛B的空间跑路器，每秒钟可以跑 2^k 千米（ k 是任意自然数）。当然，这个机器是用longint存的，所以总跑路长度不能超过maxlongint千米。小A的家到公司的路可以看做一个有向图，小A家为点1，公司为点 n ，每条边长度均为一千米。小A想每天能醒地尽量晚，所以让你帮他算算，他最少需要几秒才能到公司。数据保证1到 n 至少有一条路径。
- 100%的数据满足 $n \leq 50$ ， $m \leq 10000$ ，最优解路径长度 $\leq \text{maxlongint}$ 。

洛谷1613 跑路

- 我们对问题进行一些分析
- 题目求的是从1号点到n号点最少要几秒到达
- 我们可以看到这个跑路器，每秒跑 2^k 条边（每条边1km），所以呢，这道题目明显就和“倍增”扯上了关系。回忆倍增，我们总是用一个参数k表示 2^k
- 这道题目也一样，我们需要用一个bool类型G数组， $G[i][j][k]$ 代表从i到j是否存在一条长度为 2^k 的路径。那么如果对于任意一个k， $G[i][j][k] == \text{true}$ ，这就表示可以用1s从i到j

洛谷1613 跑路

- 再用dis数组来记录两点之间需要用多久到达。这样我们可以用G来保存所有的边，并且进行预处理，把所有一秒能到的两个点之间都连上边，并把距离相应调整为1
- 那么我们就把所有一秒能到的点之间都铺上了边，接下来我们就要求出两点之间的最短路啦，那么，大家都明白了，对于50的数据，Floyd绝对是最简单可行的办法了

洛谷1613 跑路

```
void init(){
    memset(G, false, sizeof(G));
    memset(dis, 10, sizeof(dis));
    scanf("%d%d", &n, &m);
    for(int i=1; i<=m; i++){
        int x, y;
        scanf("%d%d", &x, &y);
        dis[x][y]=1;
        G[x][y][0]=true;
        /*初始化, x到y的路径(边)最短是1, 也就是x到y存在
        一条长度为 $2^0$ 的路径(边)*/
    }
}
```

```
void work()//此函数对G和dis做预处理
{
    for(int k=1; k<=64; k++)
        //对于本题的数据,  $2^{64}$ 已经足够。
        for(int i=1; i<=n; i++)
            for(int t=1; t<=n; t++)
                for(int j=1; j<=n; j++)
                    //枚举三个点
                    if(G[i][t][k-1] && G[t][j][k-1])
                        /*如果i到t存在一条 $2^{k-1}$ 长度的路径
                        并且t到j存在一条 $2^{k-1}$ 长度的路径
                        就说明i到t, t到j都可以一秒到达,
                        路程*2刚好是2的幂, 也可以一秒到达*/
                        {
                            G[i][j][k]=true;
                            //标记从i到j存在一条长度为 $2^k$ 的路径
                            dis[i][j]=1;
                            //i到j距离可以一秒到达
                        }
}
```

洛谷1613 跑路

```
void floyd(){
    for(int k=1;k<=n;k++)
        //这里的注意点: 枚举中间点的循环放在最前面
        for(int i=1;i<=n;i++)
            for(int j=1;j<=n;j++)
                dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
        //松弛操作。
} //Floyd图论求最短路。

int main(){
    init();
    work();
    floyd();
    printf("%d",dis[1][n]);
}
```

洛谷3398 仓鼠找sugar

- 题目描述
小仓鼠的和他的基 (mei) 友 (zi) sugar住在地下洞穴中，每个节点的编号为1~n。地下洞穴是一个树形结构。这一天小仓鼠打算从他的卧室 (a) 到餐厅 (b)，而他的基友同时要从他的卧室 (c) 到图书馆 (d)。他们都会走最短路径。现在小仓鼠想知道，有没有可能在某个地方，可以碰到他的基友？
- 输入格式
第一行两个正整数n和q，表示这棵树节点的个数和询问的个数。
接下来n-1行，每行两个正整数u和v，表示节点u到节点v之间有一条边。
接下来q行，每行四个正整数a、b、c和d，表示节点编号，也就是一次询问，其意义如上。
- 100%的数据 $n \leq 100000, q \leq 100000$

洛谷3398 仓鼠找sugar

- 询问树上a到b, c到d的两条路径是否相交
- 我们容易发现, 如果相交, 记 $x = \text{lca}(a, b)$, $y = \text{lca}(c, d)$, 则必有x在c~d路径上或y在a~b路径上
- 为什么呢?
- 首先自己画一画图发现没有什么问题就可以直接用了

洛谷3398 仓鼠找sugar

- 证明一下：易知两点的lca在其路径上。如果路径相交，那么x要么在相交的路径上，要么不在。我们不妨记相交的那段为 $e \sim f$
- 如果不在，由对称性，不妨设x靠近a，那么有a到x深度递减，b到e、e到f、f到x深度递减；同样，肯定有c到f、d到e深度递减，由此可知，y必定为f，由此得证（以上的e、f和c、d的相对位置是由对称性直接设的，各位不妨画图理解一下qAq）
- 如果在的话就更好证了

洛谷3398 仓鼠找sugar

- 还剩下一件事情
- 那么如何查看一个点是否在一条路径上呢？
- 一个性质：即须满足该点到路径两端点的距离和等于两端点的距离，距离用lca和深度就可以了
- 设 $\text{lca}(u,v)=x$ ，则 u,v 间距则为 $\text{dep}[u]+\text{dep}[v]-2*\text{dep}[x]$

货车运输

- 题目描述

A国有 n 座城市，编号从1到 n ，城市之间有 m 条双向道路。每一条道路对车辆都有重量限制，简称限重。现在有 q 辆货车在运输货物，司机们想知道每辆车在不超过车辆限重的情况下，最多能运多重的货物。

- 输入格式

第一行有两个用一个空格隔开的整数 n, m ，表示A国有 n 座城市 m 条道路。接下来 m 行每行3个整数 x, y, z ，每两个整数之间用一个空格隔开，表示从 x 号城市到 y 号城市有一条限重为 z 的道路。注意： x 不等于 y ，两座城市之间可能有多条道路。

接下来一行有一个整数 q ，表示有 q 辆货车需要运货。

接下来 q 行，每行两个整数 x, y ，之间用一个空格隔开，表示一辆货车需从 x 城市运输货物到 y 城市，注意： x 不等于 y 。

- $0 < n < 10,000, 0 < m < 50,000, 0 < q < 30,000, 0 \leq z \leq 100,000$

货车运输

- 首先便是想到了Floyd的暴力方法，状态转移方程也不难推出：
 $w[i][j] = \max(w[i][j], \min(w[i][k], w[k][j]));$
- 但是复杂度太高
- 对于该题的数据范围： $0 < n < 10,000, 0 < m < 50,000, 0 < q < 30,000, 0 \leq z \leq 100,000$ 显然不可取

货车运输

- 理解一下题意，对于每次询问，找出一条路径使得这条路径上边的最小值最大
- 我们不妨先考虑哪些边是必要的（可能出现在这样的路径上）
- 最大生成树
很显然最大生成树外的边，是不会被选入题中所求路径的

货车运输

- 得到了这样一个树之后，我们便考虑如何求出两个节点之间最小边权的最大值（即为题中的最大载重），因为这两点之间的路径是唯一的，我们只需要找出这条路径便可以得到答案。
- 我们可以通过LCA来做到这一点，求LCA的方法是先从每一个根节点进行搜索，求出节点深度等信息，然后利用这些信息进行树上倍增。
- 在树上倍增的同时，可以求出边最小值的信息，从而得到答案

货车运输

```
void dfs(int node){
    vis[node]=true;
    for(int i=head[node]; i; i=edge2[i].next){ //前向星遍历
        int to=edge2[i].to;
        if(vis[to]) continue;
        deep[to]=deep[node]+1; //计算深度
        fa[to][0]=node; //储存父节点
        w[to][0]=edge2[i].w; //储存到父节点的权值
        dfs(to);
    }
    return ;
}
```

货车运输

```
//LCA初始化
for(int i=1; i<=20; i++)
    for(int j=1; j<=n; j++){
        fa[j][i]=fa[fa[j][i-1]][i-1];
        w[j][i]=min(w[j][i-1], w[fa[j][i-1]][i-1]);
    }
```


货车运输

```
int lca(int x, int y){
    if(find(x)!=find(y)) return -1; //不连通, 输出-1
    int ans=INF;
    if(deep[x]>deep[y]) swap(x,y); //保证y节点更深
    //将y节点上提到与x节点相同深度
    for(int i=20; i>=0; i--){
        if(deep[fa[y][i]]>=deep[x]){
            ans=min(ans, w[y][i]); //更新最大载重 (最小边权)
            y=fa[y][i]; //修改y位置
        }
    }
    if(x==y) return ans; //如果位置已经相等, 直接返回答案
    //寻找公共祖先
    for(int i=20; i>=0; i--){
        if(fa[x][i]!=fa[y][i]){
            ans=min(ans, min(w[x][i], w[y][i])); //更新最大载重 (最小边权)
            x=fa[x][i];
            y=fa[y][i]; //修改x,y位置
        }
    }
    ans=min(ans, min(w[x][0], w[y][0]));
    //更新此时x,y到公共祖先最大载重, fa[x][0], fa[y][0]即为公共祖先
    return ans;
}
```

POJ 3728

- Description
n个城市构成一棵树，每个城市都有一种商品有一个固定的价格，一个商人要从一个城市到另一个城市，他会在路途中选取一个城市购买这种商品然后在之后的某个城市卖掉以赚取差价，问最大差价
- Input
第一行一整数n表示城市个数，之后n个整数val[i]表示第i个城市该种商品的价格，之后n-1行每行两个整数u和v表示u城市和v城市有路径，然后输入一整数q表示查询数，每次查询输入两个整数u和v表示商人从u到v路途中可以赚取的最大差价($1 \leq n, \text{val}[i], q \leq 50000$)
- Output
对于每组查询输出一个答案

POJ 3728

- 不难发现商人要走的路线: $x \rightarrow \text{LCA} \rightarrow y$
- 可能出现的买卖情况:
 - 1. 在 $x \rightarrow \text{LCA}$ 途中完成买卖
 - 2. 在 $\text{LCA} \rightarrow y$ 途中完成买卖
 - 3. 在 $x \rightarrow \text{LCA}$ 买, 在 $\text{LCA} \rightarrow y$ 卖

POJ 3728

- 显然可以通过树上倍增来解决
- 那么我们需要维护什么值呢?
- 所以我们在线倍增维护几个值,
- $\text{Max}[i][j]$ i 到 i 的 2^j 之间的最大值
- $\text{Min}[i][j]$ i 到 i 的 2^j 之间的最小值
- $\text{Up}[i][j]$ 从 i 到 i 的 2^j 父亲之间差值最大
- $\text{Down}[i][j]$ 从 i 的 2^j , 到 i 之间的差值最大

预处理

```
void dfs(int u, int fa){
    for (int i = 0; i < g[u].size(); i++){
        int v = g[u][i];
        if (v == fa) continue;
        f[v][0] = u;
        dep[v] = dep[u] + 1;
        Max[v][0] = max(a[v], a[u]);
        Min[v][0] = min(a[v], a[u]);
        Up[v][0] = max(0, a[u] - a[v]);
        Down[v][0] = max(0, a[v] - a[u]);
        dfs(v, u);
    }
    return;
}
```

```
void init(){
    mem(f, -1);
    dep[1] = 1;
    dfs(1, 0);
    for (int j = 1; j < 20; j++){
        for (int i = 1; i <= n; i++){
            if (~f[i][j-1]){
                int a, b, k = f[i][j-1];
                f[i][j] = f[k][j-1];
                Max[i][j] = max(Max[i][j-1], Max[k][j-1]);
                Min[i][j] = min(Min[i][j-1], Min[k][j-1]);
                a = max(0, Max[k][j-1] - Min[i][j-1]);
                b = max(Up[i][j-1], Up[k][j-1]);
                Up[i][j] = max(a, b);
                a = max(0, Max[i][j-1] - Min[k][j-1]);
                b = max(Down[i][j-1], Down[k][j-1]);
                Down[i][j] = max(a, b);
            }
        }
    }
    return;
}
```

两条路径

```
int Query_up(int x, int deep, int &min_val){
    int ans = 0;
    per(i,19,-1)
    if (deep & (1 << i)){
        ans = max(ans,Up[x][i]);
        ans = max(ans,Max[x][i] - min_val);
        min_val = min(min_val,Min[x][i]);
        x = f[x][i];
    }
    return ans;
}
```

```
int Query_down(int x, int deep, int &max_val){
    int ans = 0;
    per(i,19,-1)
    if (deep & (1 << i)){
        ans = max(ans,Down[x][i]);
        ans = max(ans,max_val-Min[x][i]);
        max_val = max(max_val,Max[x][i]);
        x = f[x][i];
    }
    return ans;
}
```

计算答案

```
scanf("%d",&m);
while(m--){
    int x,y,z;
    scanf("%d%d",&x,&y);
    z = lca(x,y);
    int max_val = 0, min_val = INF,a,b;
    a = Query_up(x,dep[x] - dep[z],min_val);
    b = Query_down(y,dep[y] - dep[z],max_val);
    z = max(max(0,max_val - min_val),max(a,b));
    printf("%d\n",z);
}
```